

# Problem Set 4

## CS 3810: Algorithmic Foundations & Complexity

Prof. Elena Rostova • Department of Computer Science

Student: Alex Rivera

Student ID: 04982713

Collaborators: Sam Meridian, Jordan Vance

Submission Date: October 28, 2026

### Section 1: Dynamic Programming & Resource Allocation

#### Problem 1: Multi-Region Server Scheduling (Vertex Compute Engine)

Nexa Cloud Infrastructure operates  $n$  data centers across global regions. At the beginning of a processing cycle, we are given a set of  $n$  computational jobs  $\mathcal{J} = \{j_1, j_2, \dots, j_n\}$ . Each job  $j_i$  has a start time  $s_i$ , a finish time  $f_i$  (where  $s_i < f_i$ ), and a revenue yield  $v_i > 0$ . Two jobs  $j_i$  and  $j_k$  are *compatible* if their execution intervals do not overlap, i.e.,  $f_i \leq s_k$  or  $f_k \leq s_i$ .

- Formulate a dynamic programming state representation to compute the maximum total revenue obtainable from a mutually compatible subset of jobs.
- State and justify the recurrence relation, proving its optimal substructure property.
- Write an efficient algorithm in pseudocode and state its time and space complexity assuming jobs are pre-sorted by finish time.

#### Solution to Problem 1 (a) Dynamic Programming State Representation

Assume jobs in  $\mathcal{J}$  are sorted in non-decreasing order of finish times such that  $f_1 \leq f_2 \leq \dots \leq f_n$ .

For each job  $j_i$  ( $1 \leq i \leq n$ ), let  $p(i)$  denote the largest index  $k < i$  such that job  $j_k$  is compatible with job  $j_i$ . If no such job exists,  $p(i) = 0$ . Since jobs are pre-sorted by finish time,  $p(i)$  can be computed for all  $i$  using binary search in  $\mathcal{O}(n \log n)$  total pre-processing time.

We define the subproblem state  $DP[i]$  as the maximum revenue achievable from a mutually compatible subset chosen from  $\{j_1, j_2, \dots, j_i\}$ . The base case is  $DP[0] = 0$ , and the optimal objective value for the full set of jobs is given by  $DP[n]$ .

#### (b) Recurrence Relation & Optimal Substructure Proof

For any job  $j_i$ , an optimal selection for subproblem  $\{j_1, \dots, j_i\}$  evaluates two mutually exclusive choices:

- Case 1 ( $j_i$  is omitted):** The maximum revenue is bounded by the optimal solution for  $\{j_1, \dots, j_{i-1}\}$ , which equals  $DP[i-1]$ .
- Case 2 ( $j_i$  is included):** Revenue generated is  $v_i$ . Compatible prior jobs must finish before  $s_i$ . By definition of  $p(i)$ , the set of valid prior jobs is  $\{j_1, \dots, j_{p(i)}\}$ , yielding maximum revenue  $v_i + DP[p(i)]$ .

This establishes the dynamic programming recurrence:

$$DP[i] = \max \{ DP[i-1], v_i + DP[p(i)] \} \quad \text{for } i = 1, 2, \dots, n. \quad (1)$$

**Proof of Optimal Substructure:** Assume for contradiction that  $DP[i]$  is suboptimal, meaning there exists a compatible set  $S^* \subseteq \{j_1, \dots, j_i\}$  yielding total revenue  $R^* > DP[i]$ .

- If  $j_i \notin S^*$ , then  $S^* \subseteq \{j_1, \dots, j_{i-1}\}$ , which implies  $DP[i-1] \geq R^* > DP[i] \geq DP[i-1]$ , yielding a direct contradiction.
- If  $j_i \in S^*$ , the remaining jobs  $S^* \setminus \{j_i\}$  are mutually compatible with finish times  $\leq s_i$ . Hence  $S^* \setminus \{j_i\} \subseteq \{j_1, \dots, j_{p(i)}\}$ . The total revenue is  $v_i + R(S^* \setminus \{j_i\})$ . By optimality of  $DP[p(i)]$ , we have  $R(S^* \setminus \{j_i\}) \leq DP[p(i)]$ . Thus  $R^* \leq v_i + DP[p(i)] \leq DP[i]$ , contradicting  $R^* > DP[i]$ .

#### (c) Pseudocode & Complexity Analysis

Below is the complete implementation with full path reconstruction.